

# **ESP32-based Smart RC Car using ESP-NOW (Automotive Lighting Automation)**

## **PROJECT REPORT**

Prepared By: Lovedeep Singh

Degree: [B.Sc.](#) Non-Medical (Science)

Domain: Embedded Systems | IoT | Automotive Electronics

Report Date: May 2026

## Declaration

I am Lovedeep Singh, my project report title is “ESP32-based Smart RC Car using ESP-NOW (Automotive Lighting Automation)”. I declare that it is an original work done by me for academic and learning purposes. All provided information and implementation in this report is based on my real life experiences and standard technical references.

Name: Lovedeep Singh

Signature:- 

Date: May 2026

## **Acknowledgement**

I would like to thank my teachers, mentors, and online technical supporters for providing valuable learning resources. I also acknowledge the official ESP32 documentation and embedded system references that help me to complete my project.

## **Abstract**

This project represents an ESP32-based Smart RC Vehicle, which stimulates realistic automotive control experience using embedded systems and wireless communication. With the help of ESP32-NOW(communication) I was able to have a low-latency wireless operation on the Wi-Fi hardware of ESP32. A transmitter ESP32 reads buttons, throttle and steering readings from installed joysticks and receiver ESP32 gets real-time packets of information to function servo motors, lights and (ESC) on the vehicle.

The receiver ESP32 gives commands to the servo motor and (ESC) to control steering and speed. In addition to movement, the system has lighting functions such as headlight control on transmitter, hazard mode, brake light automation, and turn signals with auto off logic. Trim calibration is possible due to EEPROM storage on both transmitter and receiver used to correct joystick centering. To improve stability, the receiver includes steering center auto-calibration. A failsafe mechanism has been implemented to stop the vehicle during communication loss. The project shows practical knowledge of embedded systems, C++ programming, real-time logic, wireless communication and automation.

## **Table of Contents**

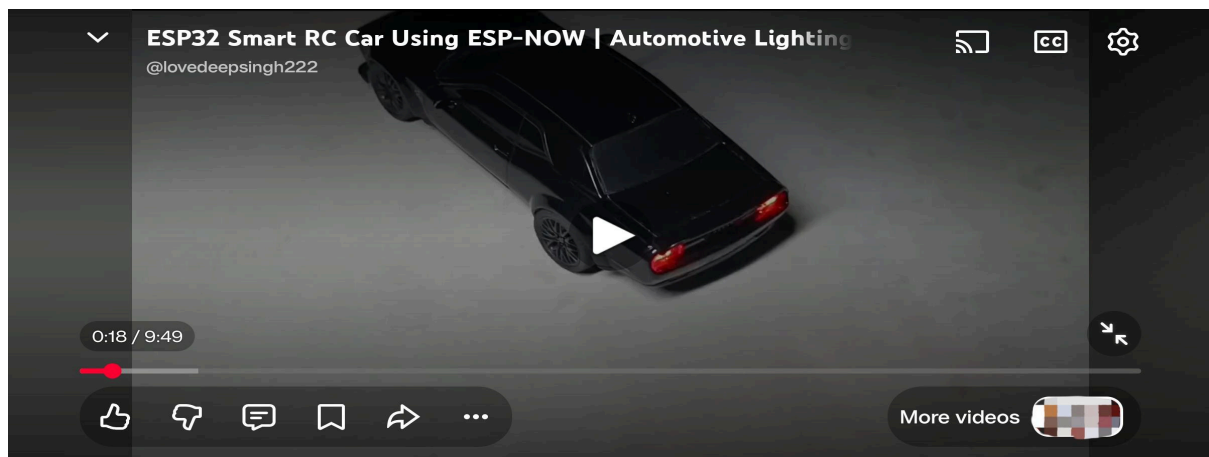
1. Introduction
2. Youtube link for demo video
3. Codes link for receiver and transmitter
4. Objectives
5. System Overview
6. Hardware Requirements
7. Software Requirements
8. Communication Method (ESP-NOW)
9. Pin Configuration
10. Circuit Connections and diagrams
11. Implementation Details
12. Control Logic and Automation Features
13. Testing and Results
14. Challenges Faced
15. Conclusion
16. Future Scope
17. References

## 1.Introduction

In the modern world, embedded systems are widely used in new automotive electronics for automation, safety and control through wire and wireless. Normal RC toys mainly have basic throttle and steering systems. This project was introduced to replicate realistic automotive control behavior and lighting stimulation with safety logic.

The ESP32 microcontroller was selected because it is powerful due to dual core processor, built-in wireless communication (ESP-NOW), wifi, bluetooth support and 11 to 18 safe usable pins. These factors are important to bring future upgrades.

## 2. Youtube link for demo video

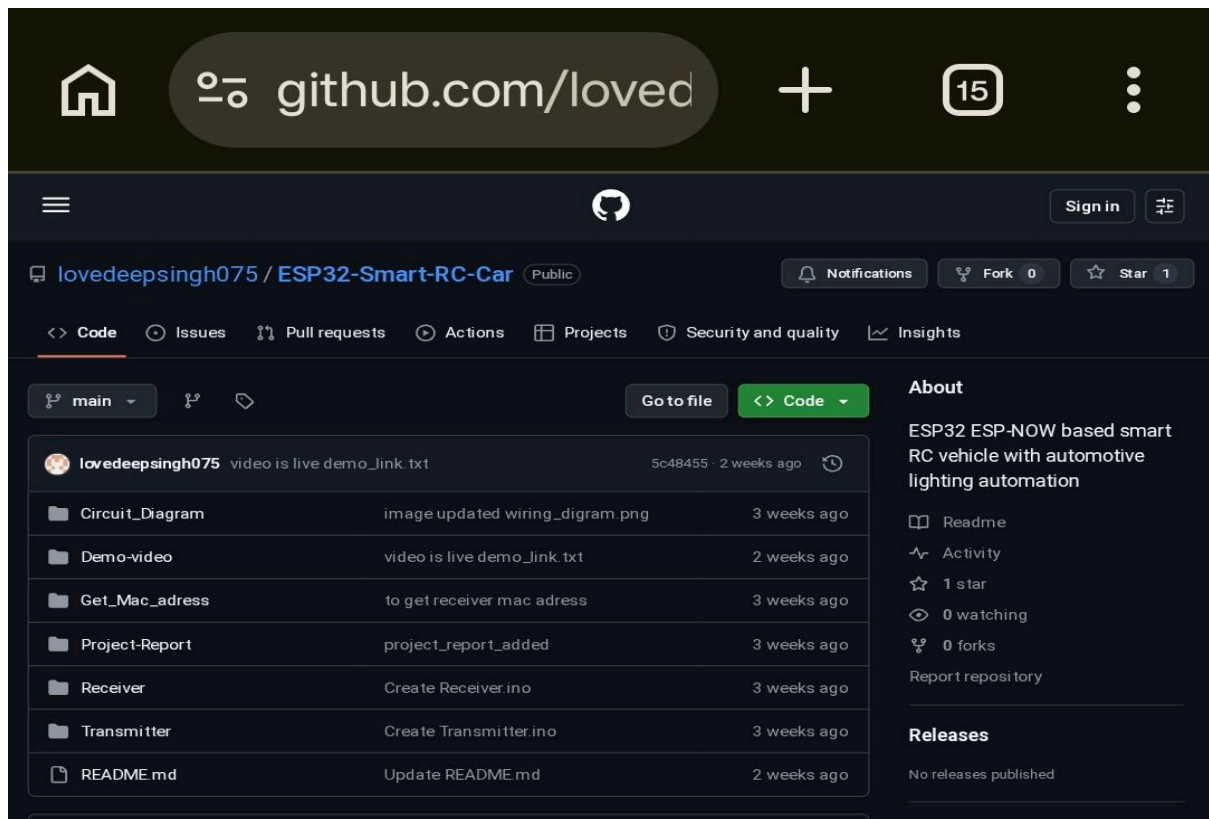


<https://youtu.be/zxEzSVyY-x0?si=88oKbUOQbi0de5zX>

Click above link for demo video (Uploaded and Created by me on my personal channel).



### 3. Codes link for receiver and transmitter



<https://github.com/lovedeepsingh075/ESP32-Smart-RC-Car.git>

Click above link for codes and further details.

### 4. Objectives

Develop low-latency wireless control using ESP-NOW.

Implement servo steering and ESC throttle control.

Implement headlights, hazard mode, indicators, and brake light automation.

Implement indicators memory logic.

Implement trim calibration using EEPROM.

Implement failsafe mode during communication loss.

Improve stability using steering center auto-calibration.

## 5. System Overview

The project consists of two ESP32 modules:

Transmitter Unit:

Reads joysticks readings for throttle and steering and buttons for headlight and hazard, applies trim calibration and transmits control packet using ESP-NOW technology.

Receiver Unit:

Receiver when received information packet by transmitter side controls steering through servo motor and ESC throttle and automotive lighting functions.

## 6. Hardware Requirements

Transmitter:

ESP32 Dev Bord

Two 1-axis joystick modules

6 buttons (4 trim + headlight + hazard)

TP4056 Charging Module

Main on/off switch

Receiver:

ESP32 Dev Board

Servo motor (steering)

ESC + DC motor( instead: *you can use modified servo motor also*)

Some LEDs for headlights(white), indicators(yellow), break light(red)

TP4056 Charging Module

Main on/off switch

**NOTE:** All modules must share a common ground.

Battery for both receiver and transmitter (use at least 2000mAh for smooth operation)



# HARDWARE REQUIREMENTS

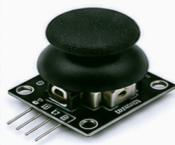
## TRANSMITTER COMPONENTS



ESP32 Dev Board  
(1 No.)



1-Axis Joystick Module  
(Throttle)  
(1 No.)



1-Axis Joystick Module  
(Steering)  
(1 No.)



Push Buttons  
(6 Nos.)



18650 Battery  
(3.7V)  
(1 No.)



TP4056 Charging  
Module  
(1 No.)

## RECEIVER COMPONENTS



ESP32 Dev Board  
(1 No.)



Servo Motor  
(SG90)  
(1 No.)



ESC (30A)  
(1 No.)



DC Motor (775)  
(1 No.)



LEDs (5mm)  
(Red, White, Yellow)  
(6 Nos.)



3.7V 2000mAh Li-ion  
Battery  
(1 No.)



Battery Holder /  
Switch  
(1 No.)



Jumper Wires  
(As required)



Chassis with Wheels  
(1 Set)

## 7. Software Requirements

Arduino IDE

Embedded C++

Libraries: `espa_now`, WiFi, EEPROM, ESP32Servo (recommended by expressive system)

## 8. Communication Method (ESP-NOW)

ESP-NOW provides low-latency in wireless communication. It uses ESP32 Wi-Fi radio but does not require any router or internet connection. All these result in reliable real-time control behaviour.

## 9. Pin Configuration

Receiver ESP32:

Steering Servo: GPIO 25

ESC: GPIO 26

Headlight: GPIO 4

Front Left: GPIO 13

Front Right: GPIO 15

Rear Left: GPIO 12

Rear Right: GPIO 14

Brake Light: GPIO 27

Transmitter ESP32:

Throttle Analog: GPIO 32

Steering Analog: GPIO 33

Trim Throttle Up: GPIO 19

Trim Throttle Down: GPIO 18

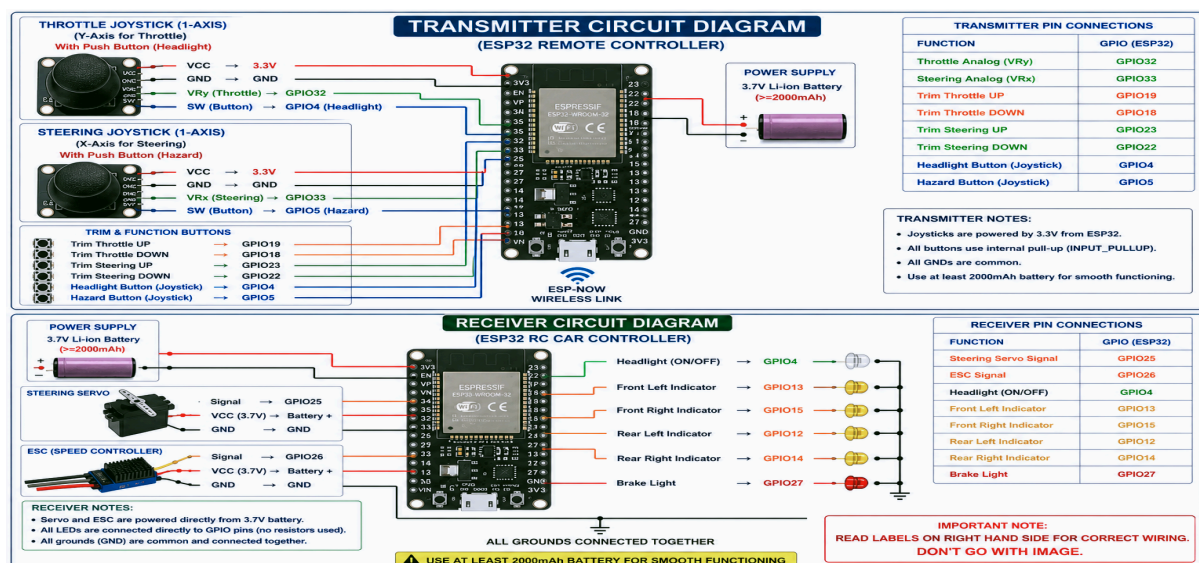
Trim Steering Up: GPIO 23

Trim Steering Down: GPIO 22

Headlight Button: GPIO 4

Hazard Button: GPIO 5

## 10. Circuit Connections and diagrams



\*Read notes before proceeding wiring.

## **11. Implementation Details**

Transmitter firmware reads analog values and transmits digital values, trim adjustments are stored in EEPROM of steering and throttle, and sends packets continuously (assist fail safe).

Receiver firmware decodes packet and drive servo and ESC, controls lighting logic, performs failsafe when needed.

## **12. Control Logic and Automation Features**

Headlights: simply toggled from the transmitter.

Hazard: blinks all indicators and overrides turn signals.

Turn Indicator: triggered automatically when steering take turn.

Indicator memory: continues blinking for 1.5 second after steering returns to center automatically.(Separate indicators used in front and in rear indicators used rear light when needed).

Brake Light: turn ON during each sudden throttle release or stop.

Break Light as reverse light: when value goes slightly below stop( vehicle moves in reverse direction) break light stimulates reverse light by continuing flashing.

Rear lights as indicators for rear: either headlight is on or off indicators over right rear light to indicate turn. In other words, if steering is turned in left, rear left light will work as indicator and right will remain on/off toggled with headlights.

Failsafe: In condition, loss communication(packet does not reach) vehicle stops.

## **13. Testing and Results**

The system was well tested:

Steering/throttle response, headlight toggle, hazard blinking, indicators logic, brake light automation, EEPROM trim saving and failsafe.

Results confirmed stable performance (above video is proof)

## 14. Challenges Faced

Challenges to stabilise power, servo jitter(related to power issue), calibration of excess throttle cause drift, mechanical alignments,

I tackled with power problem by using 2000mAh battery in both transmitter and receiver, servo jitter solves automatically onwards, mapping joystick correctly(after finding real values of joystick through a code)

## 15. Conclusion

This project successfully represents a real car using ESP32-based Smart RC Vehicle implementing automotive-inspired lighting automation and safety features. ESP-NOW communication helps to have smooth real time control.

## 16. Future Scope

Wheel encoder feedback

Installing a horn

Mobile connectivity

GPS tracking

Obstacle avoidance system

Car health monitoring

## 17. References

1. <https://randomnerdtutorials.com/esp32-pinout-reference-gpios/>  
Used for understanding ESP32 GPIOs, ESP-NOW and PWM control.
2. <https://share.google/d1amGaqBILeV3cDws>  
Used to share my project, provides a platform where I can upload my whole project.
3. <https://randomnerdtutorials.com/esp-now-esp32-arduino-ide>  
Helped in understanding ESP-NOW pairing and data transfer.
4. [https://github.com/proknowdiy/3ch\\_esp-now\\_transmitter\\_and\\_receiver](https://github.com/proknowdiy/3ch_esp-now_transmitter_and_receiver)  
Used as initial learning reference for ESP-NOW transmitter and receiver communication.
5. Youtube tutorials and online electronics forums were also referred to for troubleshooting ESP32 connections, ESC calibration and servo control.